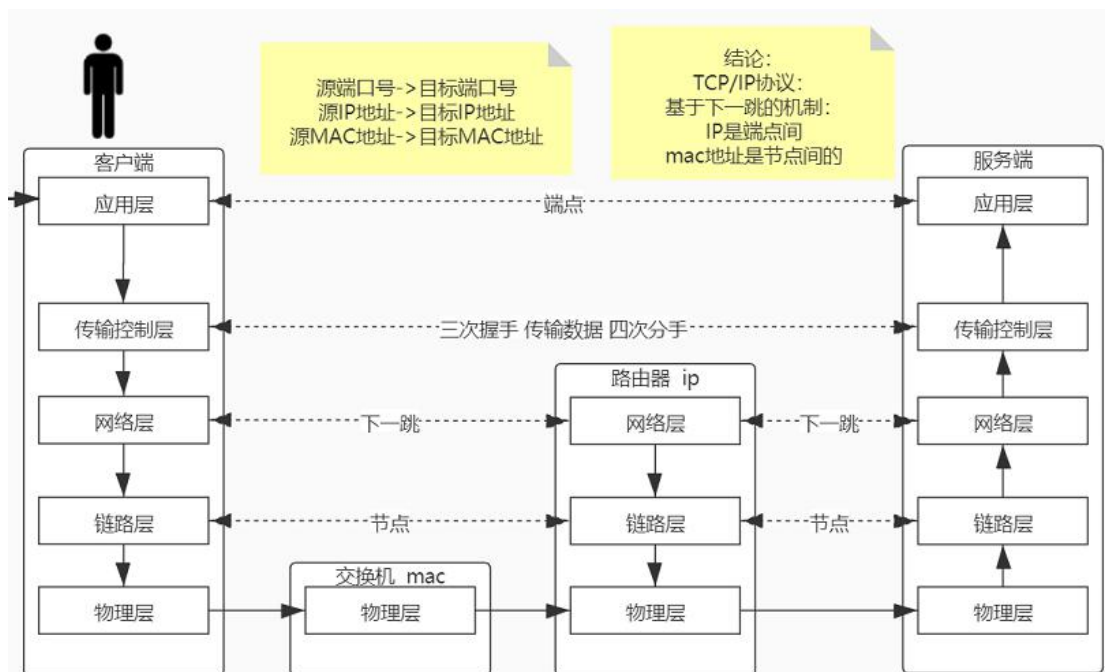


多线程笔记整理

一、概述

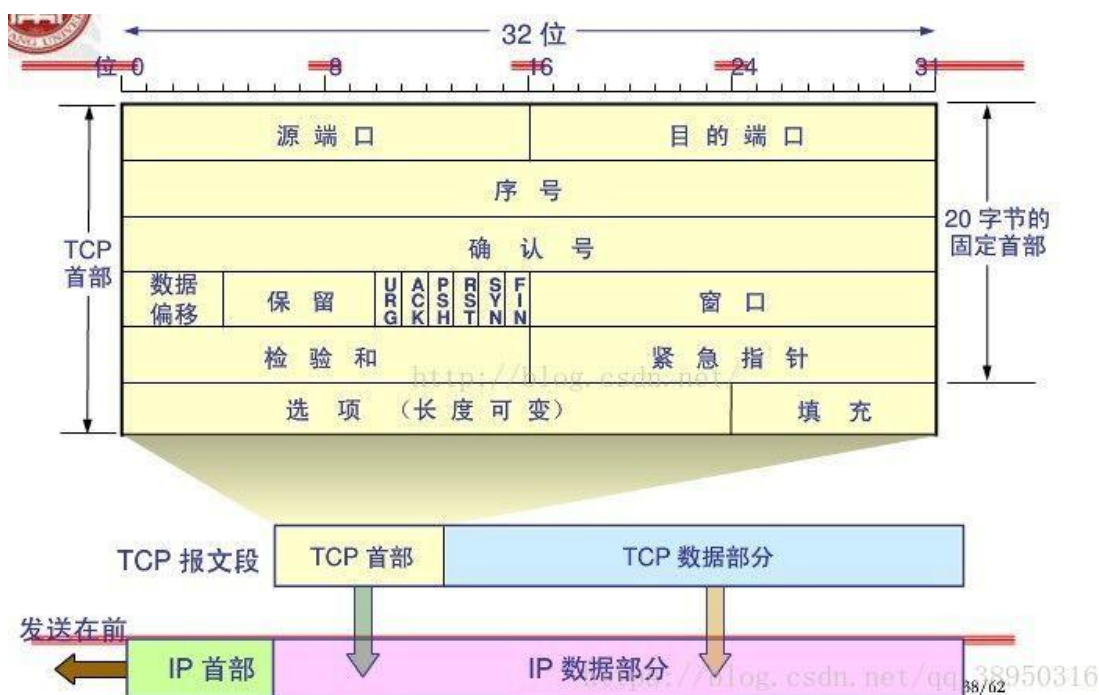
- 1) 网络应用之间的通信一般需要明确 IP 和端口，因为一个网络应用往往占用一个或多个端口，然后采取某种网络协议来完成通信。
- 2) 网络编程一般使用 socket 类又名套接字，以 BIO 或 NIO 的形式，常用 BIO，NIO 常用于线程并发的编程中
- 3) 常见的协议及区别
 - 1). UDP 协议
传递数据时不关心连接状态直接发送,他的传输效率高,传递的数据容易丢包
 - 2). TCP/IP 协议
传递数据时关系连接的状态,连接成功才会发送数据,他的传输效率相对于 UDP 协议要低,会通过三次握手机制保证数据的完整性, java 编程默认使用的协议
 - 3). Http 协议
相对 TCP 协议效率较低，是对 TCP 协议的封装
- 4) http/socket/tcp-ip 通信详解（重点）
 - a) 网络传输的七层协议是概念，实现的时候将其合并为以下 5 层（这五层要被下来）。
 - b) 最上层的应用层是可以实现多个自定的协议（如 http 协议，ftp 协议等），所有上层的协议都是对下层协议接口的调用实现的。如传输控制层有 TCP 和 UDP，http 其实就是对 TCP 的封装。



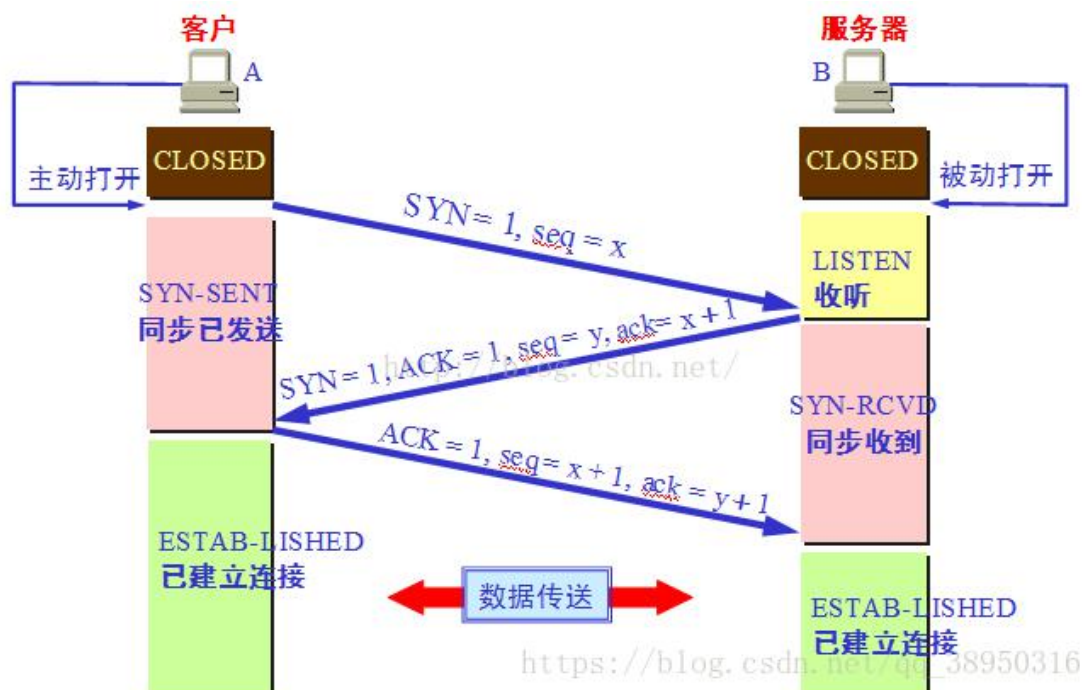
- a) HTTP 以及 socket 通信过程分析
- b) 首先我们在发起一个 HTTP 请求时,应用层会用 socket 对象来包装我们的请求信息，然后调用传输控制层的 TCP/IP 协议接口，与服务端发生三次握手确认连接可用。
- c) TCP 的三次握手

[1] 首先就是 TCP 包的三个重要的标志符（只有 0/1 两种状态）

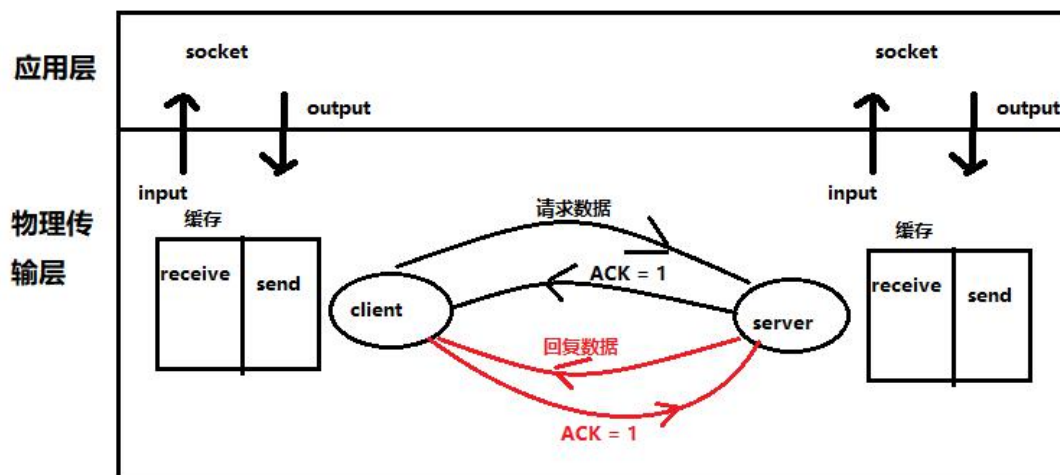
- ACK: 确认是否有效
- SYN: 请求建立连接
- FIN: 希望断开连接



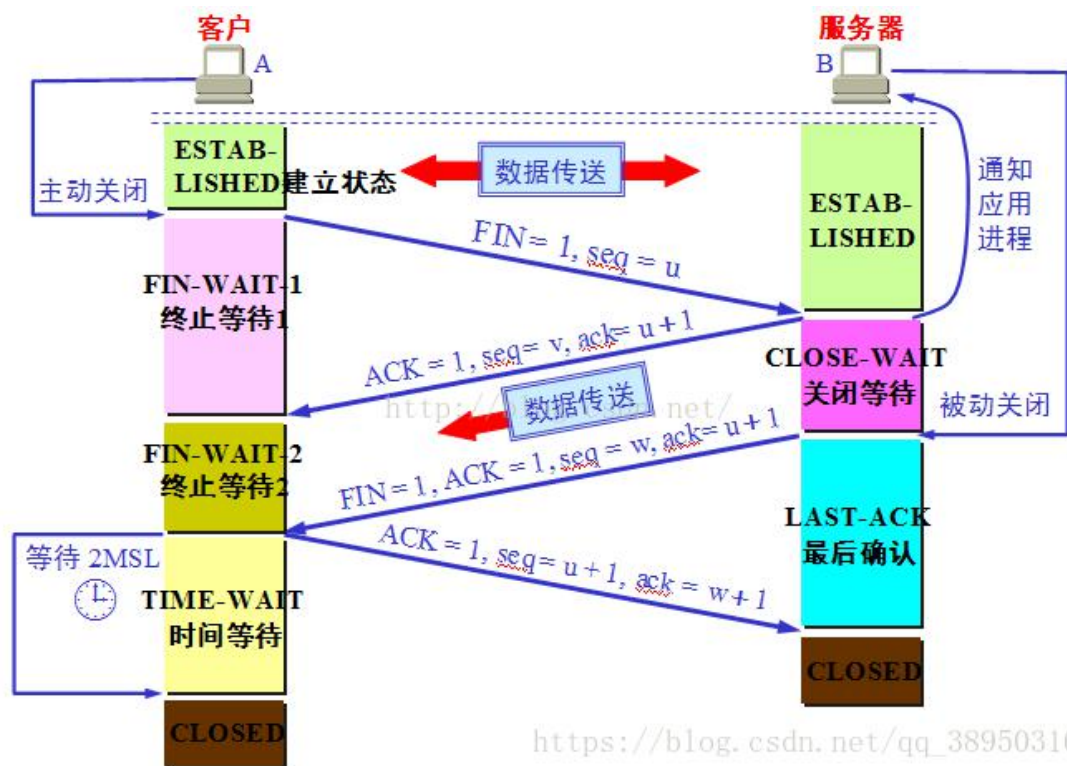
[2] 客户端传输控制层会打一个包，将 SYN 位置为 1，请求链接；服务端会收到这个包，如果可以进行消息接受，就会回复一个包，将 SYN 置为 1，同时将 ACK 置为 1，表示同意建立链接；然后客户端又会回复一个包，将 ACK 置为 1，表示同意建立连接。三次握手执行完毕后，不管服务端是否已经 accept，双方就会在各自的内存中开辟一块缓存用来存放 receive 数据和 send 数据。



- [3] 建立连接并开辟了缓存之后,就可以就行数据传输了。我们会把发出的消息和收到的消息用 socket 对象的输入流 (InputStream) 和输出流 (OutputStream)来包装,应用层的输出流 (OutputStream) 会被压入开辟的缓存的 Send 队列中,由网卡发送给对方,同时会接受对方发来的包,ACK 置为 1,表示确定收到了消息,对方发来的消息会存入缓存中的 receive 队列中,由 socket 对象取出,传至应用层 (InputStream),这样的话就完成了数据的传输,其实就是向缓存中压入数据或者取出数据。



- d) 当通信完毕后,应用层执行 socket 关闭方法,物理传输层会进行四次挥手来断开连接。
- [1] 客户端向服务端发送一个包,将 FIN 置为 1,表示想断开连接;
 - [2] 服务端向客户端发送一个包,将 ACK 置为 1,表示确定收到了消息
 - [3] 服务端把剩余的业务处理完后,将 FIN 置为 1,ACK 置为 1,表示消息处理完了,可以断开连接;
 - [4] 客户端向服务端发送一个包,将 ACK 置为 1,表示收到了服务端的断开请求,链接断开,同时缓存清空。

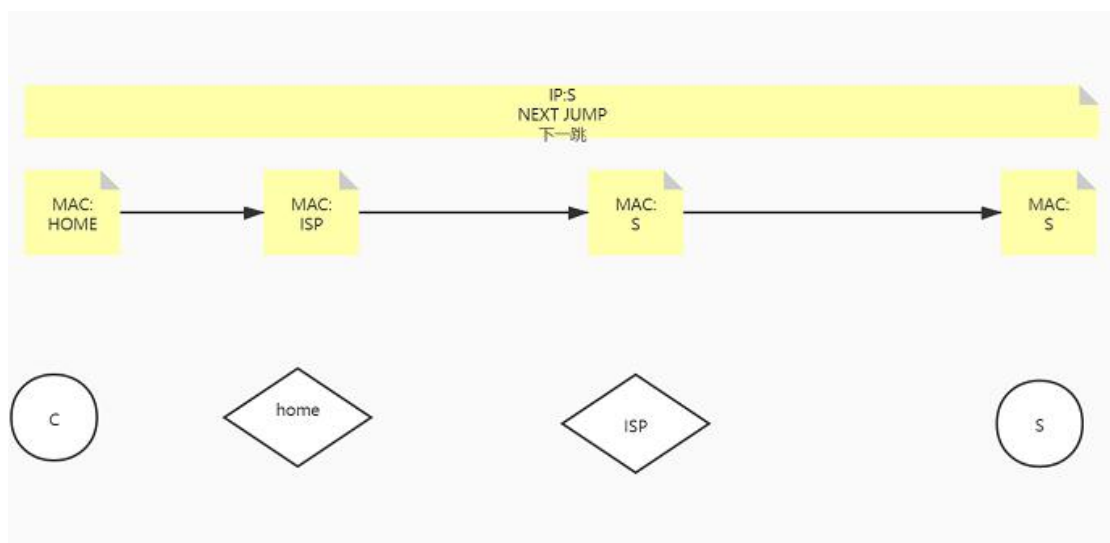


e) 一些基础知识

- [1] Socket 套接字是客户端 ip 和端口+服务端 ip 和端口组成的全球唯一的键值对，只要保证套接字的唯一，就能进行数据传输，也就是说一个服务端口可以对多个客户端进行服务，一个客户端可以同时访问多个服务端。

f) 然后我们对下面几层进行一些说明（网络层，链路层，物理层）

- [1] 我们的因特网可以看作是由路由器，路由总站组成的一层又一层的局域网；
- [2] 网络层做的是 IP，掩码等信息的分配，路由表的存储
- [3] 链路层存着下一个路由器的信息，然后拿着网络层传来的 IP,目标地址，消息，根据路由表映射关系，通过物理层，把信息传递至下一个路由节点；
- [4] 然后下一个路由节点，链路层又根据它的路由表映射关系，将消息传给下一个节点
- [5] 直到链路层的地址和目标地址一致，才能完成消息的接受。



- 5) 我们可以通过 new ServerSocket + 端口的形式创建一个服务对象，通过 new Socket + 服务 IP,端口的形式创建一个连接对象，并通过输出流 OutputStream 向服务对象发送信息，通过捕获输入流 InputStream 来获取服务器的信息；服务对象通过 accept 方法来获取连接对象；通过捕获连接对象中的输入流 InputStream 来获取，客户端的信息，通过输出流 OutputStream 向客户端发送消息。

```
public static void main(String[] args) throws IOException {
```

```
//创建socket对象      创建连接对象
Socket socket = new Socket( host: "192.168.43.81", port: 8989);
```

```
//向服务器发送数据
OutputStream outputStream = socket.getOutputStream();
outputStream.write("hello Server".getBytes());
//代表客户端发送的数据完成
socket.shutdownOutput(); 向服务器发送消息
```

```
//获取服务器端响应数据
InputStream inputStream = socket.getInputStream();
int len = 0;
byte[] b = new byte[1024];
StringBuilder builder = new StringBuilder();
while(true){
    len = inputStream.read(b); 接受服务器消息
    if(len== -1) {break;}
    builder.append(new String(b, offset: 0, len));
}
System.out.println(builder.toString());
```

```
socket.shutdownInput();//确定读取响应数据结束
```


// 创建一个服务器

```
ServerSocket serverSocket = new ServerSocket(port: 8989);  
System.out.println("服务器已经启动.....");  
while(true) {
```

// 让服务器接受|接收客户端

```
Socket socket = serverSocket.accept();
```

创建服务器

获取连接对象

// 处理请求数据

```
InputStream inputStream = socket.getInputStream();  
StringBuilder builder = new StringBuilder();  
int len = 0;  
byte[] b = new byte[1024];  
while (true) {  
    len = inputStream.read(b);  
    if (len == -1) {break;}  
    builder.append(new String(b, offset: 0, len));  
}
```

获取连接对象
中的消息

```
System.out.println(builder.toString());  
socket.shutdownInput();// 获取请求数据 结束
```

// 处理业务

// 服务端响应客户端数据

```
OutputStream outputStream = socket.getOutputStream();  
outputStream.write("讲".getBytes());  
socket.shutdownOutput();
```

向连接对象
发送消息

- 6) 在网络编程中，ServerSocket 如果没有并发操作，那么每次只有一个线程，必须等待获取的套接字对象操作完，才能对下一个请求进行操作，这样大量的请求都会处于等待状态，很不利于生产开发。

```

public static void main(String[] args) throws IOException {
    // 创建一个服务器
    ServerSocket serverSocket = new ServerSocket( port: 8989);
    System.out.println("服务器已经启动.....");
    while(true) {
        // 让服务器接受|接收客户端
        Socket socket = serverSocket.accept();
        // 处理请求数据
        InputStream inputStream = socket.getInputStream();
        StringBuilder builder = new StringBuilder();
        int len = 0;
        byte[] b = new byte[1024];
        while (true) {
            len = inputStream.read(b);
            if (len == -1) {break;}
            builder.append(new String(b, offset: 0, len));
        }
        System.out.println(builder.toString());
        socket.shutdownInput();// 获取请求数据 结束
        // 处理业务
        // 服务端响应客户端数据
        OutputStream outputStream = socket.getOutputStream();
        outputStream.write("讲".getBytes());
        socket.shutdownOutput();
    }
}

```

获取的套接

操作

必须等套接字操作完，释放掉，才能捕获下一个套接字

二、 HTTPS 和 HTTP

1. 对称加密

(1) 原理

加密和解密使用同一个密钥，且加密算法和解密算法互为逆向（如乘法和除法）

$$\begin{array}{ccc}
 \text{A、B、C} & \xrightarrow{+3} & \text{D、E、F} \\
 65、66、67 & \xleftarrow{-3} & 68、69、70
 \end{array}$$

https://blog.csdn.net/qq_23095607

(2) 常见使用方式

发送方：对数据每个字符的 ascii 码，使用密钥结合加密算法得到密文

接收方： 使用解密算法，结合密钥对密文解密，得到数据

常见的对称加密有：AES，DES，Base64 加密等

(3) 缺陷

如果在网络传输的话很容易被劫持，由于原理简单，即使没有劫持到密钥，普通计算机24H 以内也可破解

2. 非对称加密

(1) 原理

非对称加密是根据加密和解密算法，会生成公钥和私钥；

被公钥加密后，密文无法通过公钥解密，只能使用私钥解密。

同样，被私钥加密后，也只能通过公钥解密；

$$m^e \text{ Mod } N \equiv c \quad \text{公钥: } (e, N)$$

$$c^d \text{ Mod } N \equiv m \quad \text{私钥: } (d, N)$$

m 为原文， c 为密文

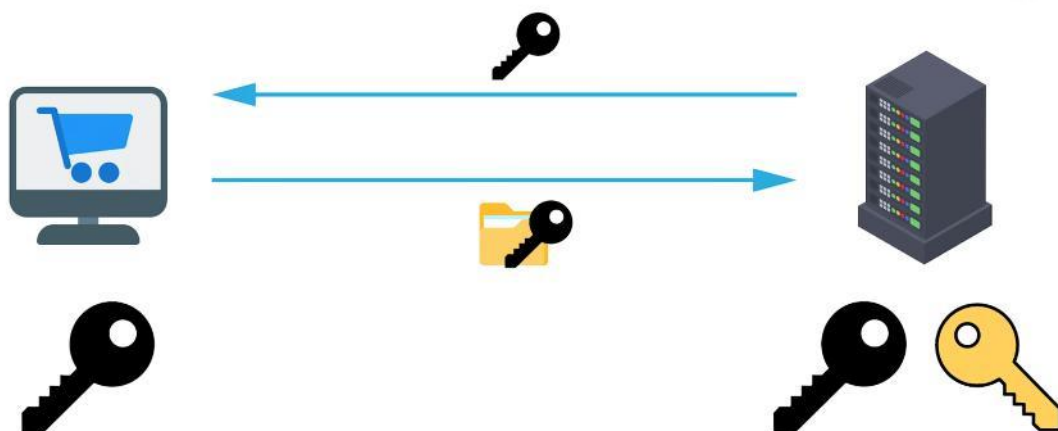
https://blog.csdn.net/qq_23095607

(2) 常见的使用方式

常见的非对称加密的方式为：RSA

首先会根据加密算法，生成公钥和私钥，私钥存储在服务器，公钥则可以通过网络等形式存储在客户端；

客户端在发送消息时，使用公钥加密消息，服务器收到消息使用私钥解密，读取内容；这样，即使公钥被截获了，信息依然是安全的；



(3) 非对称加密的缺陷

① 中间人攻击

很多情况下，公钥是服务器通过网络传输给客户端，客户端使用公钥给数据加密后再发送给服务器；

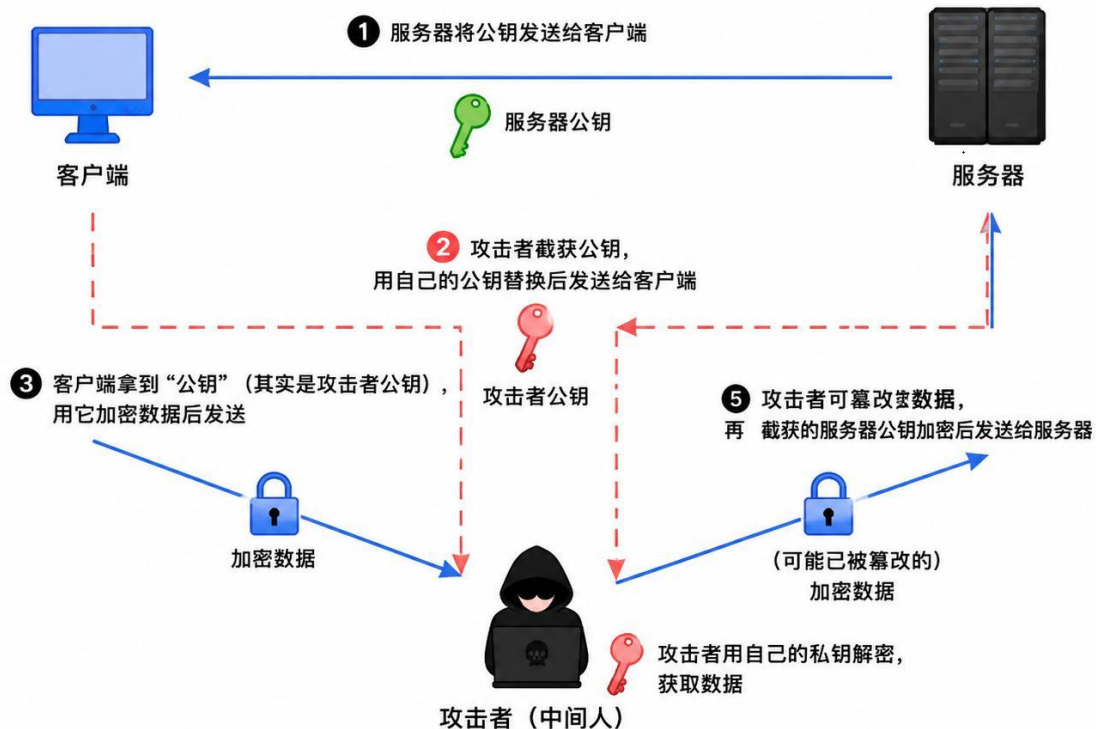
如果在通信过程中有人要截获消息，并且截获方也生成了一对公钥和私钥

服务器给客户端发送的公钥，被截获并替换为截获方的公钥；

客户端拿到截获方的公钥后，用它对消息进行加密并发送；

截获方拿到客户端的消息后，用自己的私钥解密，即可读取消息内容

截获方还可以篡改客户端的消息，再用截获的服务器公钥加密发送给服务器。



3 . HTTPS

HTTPS 是 HTTP 和 SSL/TLS 技术的结合，可以最大限度的解决中间人攻击的问题；

SSL/TLS 依赖于 CA 证书来实现信息安全

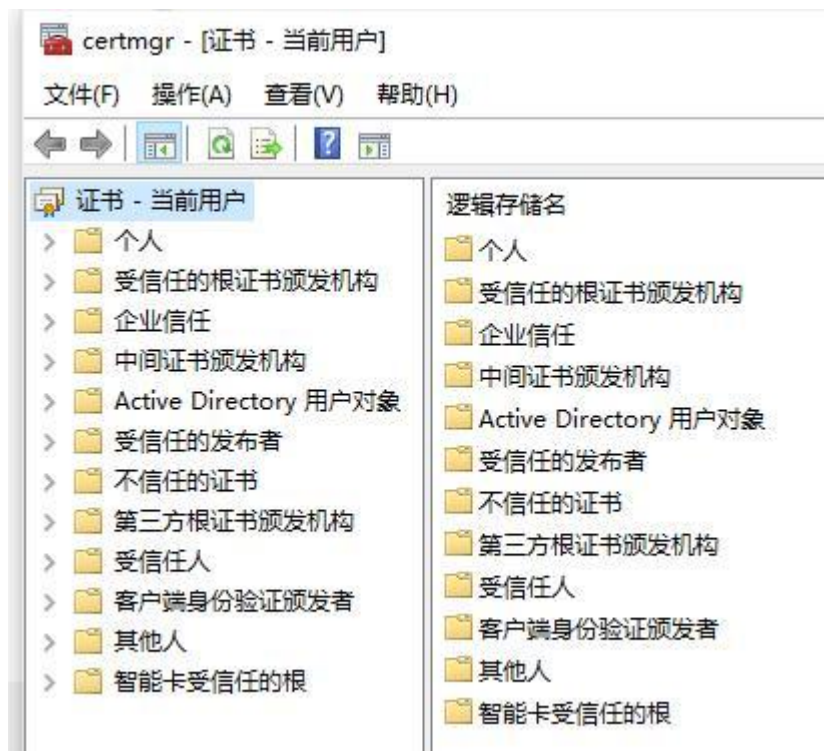
(1) CA 证书申请

中间人攻击的根本问题是，公钥以明文的形式在网络中传输，容易被截获方替换更改。

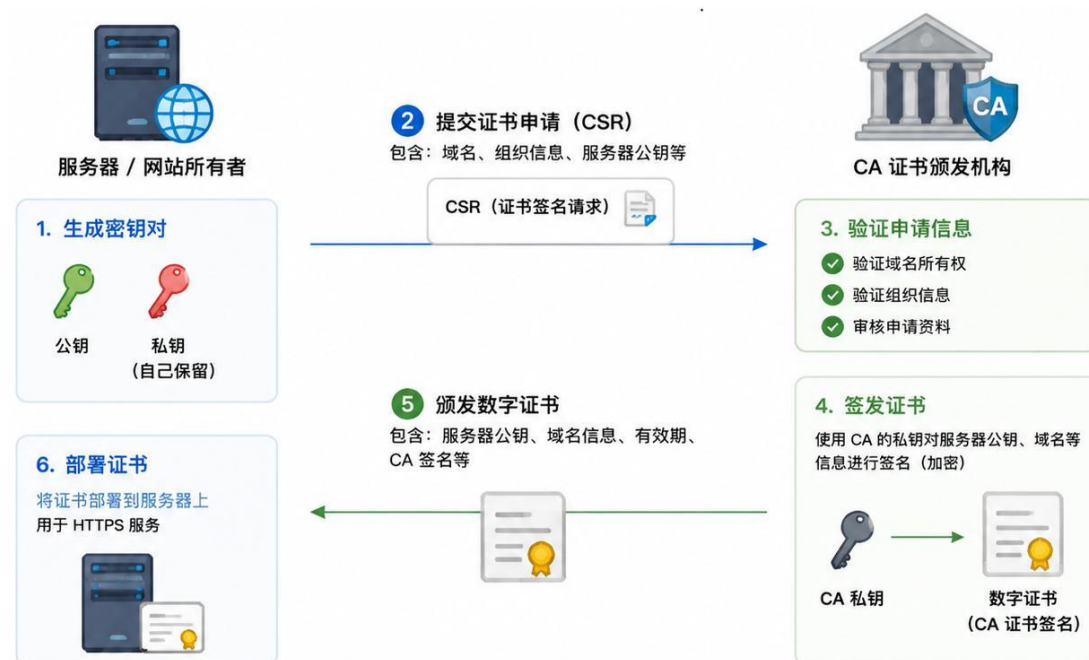
为了解决这个问题，需要一个受信任的第三方平台，对公钥进行加密，具体做法为：

① 在申请域名的时候，CA 证书颁发机构会生成一对公钥和私钥；

通常 CA 证书颁发机构的公钥，对于所有域名都是一样的，随浏览器或操作系统的安装预存在本地。



② CA 证书颁发机构会用它的私钥，将服务器的公钥、域名等信息进行签名,CA 证书签名、服务器公钥、域名等信息保存在 CA 证书中，CA 证书保存在服务器中；



4 . HTTPS 通信

- (1) 首先客户端向服务器发起 HTTPS 请求
- (2) 服务器将证书发送给客户端，证书中有服务器公钥、域名等信息，以及 CA 证书签名



(3) 客户端使用预存的 CA 证书公钥进行解密，校验域名等信息，并校验 CA 证书签名，确保服务器公钥可信；

(4) 在确保服务器公钥可信后，客户端会生成对称加密 AES 的密钥，用服务器公钥对 AES 密钥加密，并发送给服务器；

(5) 服务器用私钥解密，获取到 AES 密钥

(6) 这样，客户端每次在和服务器通信时，都会生成随机的对称加密密钥

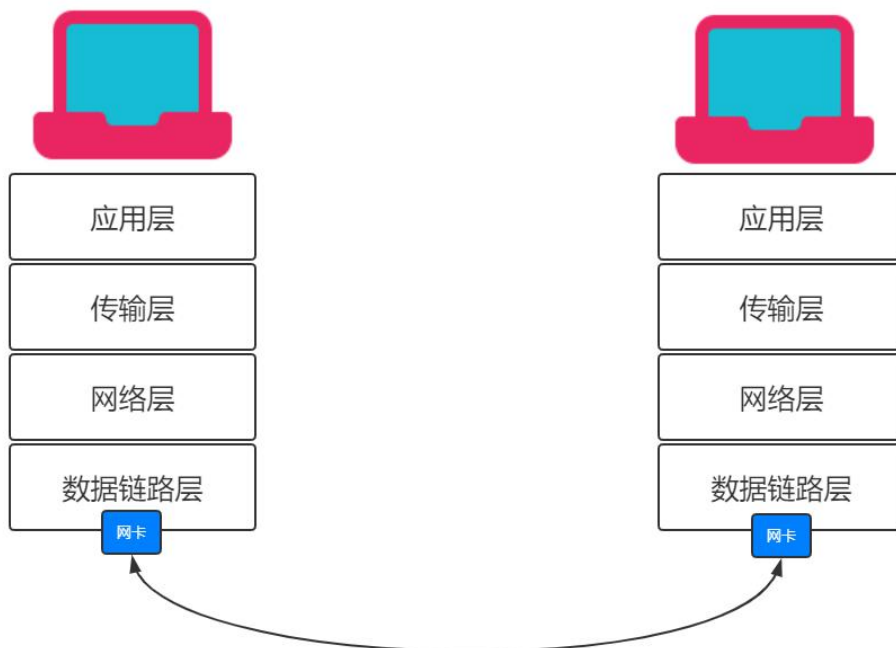
(7) 之后，服务器和客户端通信，只需要使用对称加密即可，提高效率；

5. 不可逆加密

常见的不可逆加密：MD5，MD5 是一种固定的 hash 算法，理论上不可逆，但是存在 hash 碰撞，所以可以预测

二、网卡和 IP 详解

1. 网卡



操作系统默认使用物理网卡，每一个物理网卡都有唯一的 MAC 地址，也可以通过机制创建多张虚拟网卡，如在创建虚拟机、安装 Docker 时，操作系统就会为每一个虚拟机创建

专属虚拟网卡，虚拟网卡依通过桥接物理网卡等形式工作。
每个网卡可以配置多个 IP，每个 IP 的端口上限是 65535



2 . IP、网段、掩码

- (1) IP
用来标记网络的唯一性标志，是 32 位的二进制数，分 4 段，在日常使用中以 10 进制表示；
- (2) 子网掩码
区分 IP 地址中的网络部分和主机部分，网络部分即常说同一个子网等
同样是 32 位的二进制数，分 4 段，在日常使用中以 10 进制表示



与 IP 进行与运算，如果相同则表示处于同一个网络部分，即网段



常见的子网掩码，将网络部分标记为 255，将主机部分标记为 0，便于区分

掩码	IP	网络位	表示
255.0.0.0	192.168.1.1	192	192.168.1.1/8
255.255.0.0	192.168.1.1	192.168	192.168.1.1/16
255.255.255.0	192.168.1.1	192.168.1	192.168.1.1/24

(3) 网段

网络部分一样的 IP 地址，称之为同一个网段

(4) 网络地址

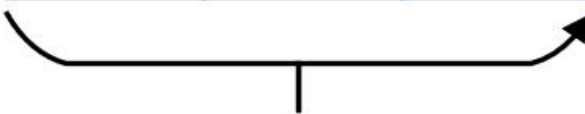
在一个网段中最小的 IP 地址，即网络部分相同，主机部分为 0

通常后面还要写明网络部分在二进制表示时的位数，为 8 的整数倍

子网掩码：255.255.255.0

192	168	10	0
-----	-----	----	---

11000000	10101000	00001010	00000000
----------	----------	----------	----------



网络部分：3×8=24

网络地址：192.168.10.0/24

(5) 广播地址

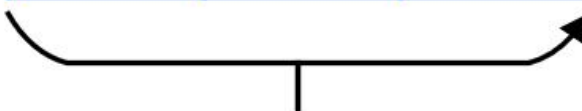
在一个网段中最大的 IP 地址，即网络部分相同，主机部分为 255

通常后面还要写明网络部分在二进制表示时的位数，为 8 的整数倍

子网掩码：255.255.255.0

192	168	10	255
-----	-----	----	-----

11000000	10101000	00001010	11111111
----------	----------	----------	----------

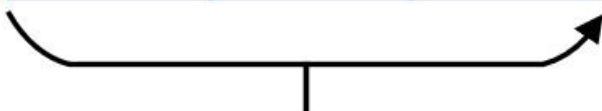
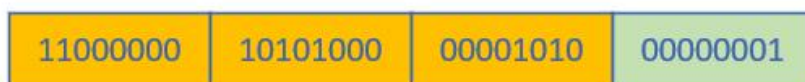
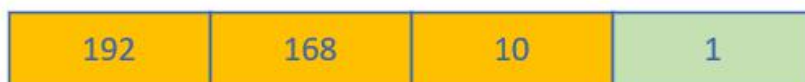


网络部分：3×8=24

网络地址：192.168.10.255/24

网络地址和广播地址不可以作为 IP，可用 IP 必须位于网络地址和广播地址的开区间内

子网掩码：255.255.255.0



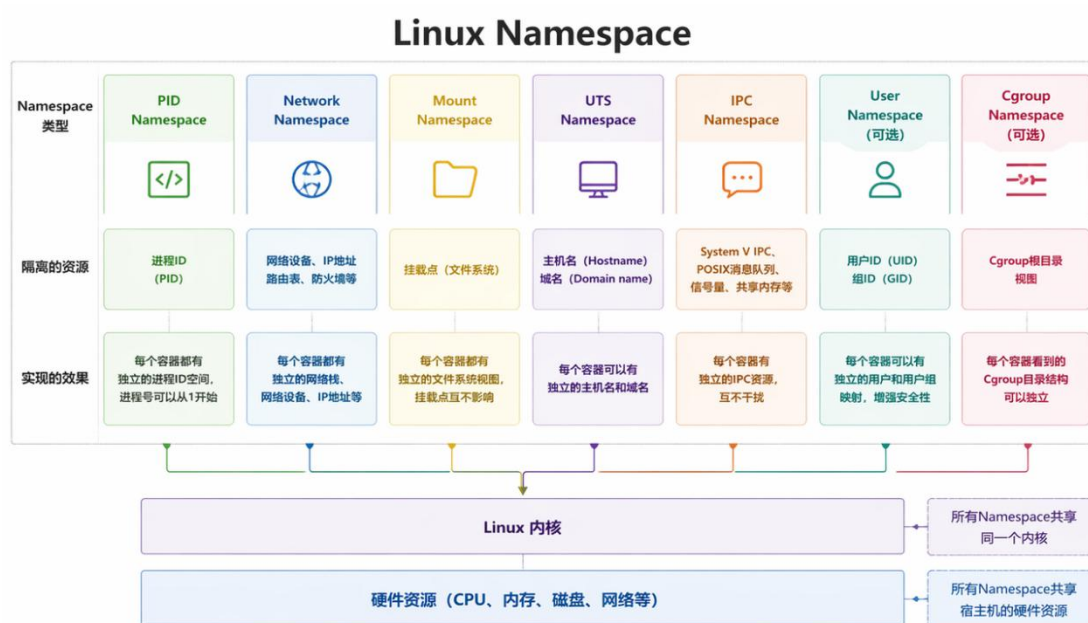
网络部分：3×8=24

最小可用地址：192.168.10.1/24

三、Linux 网络知识

1. Namespace

Linux Namespace 是 Linux 提供的一种内核级别环境隔离的方法。可以用来隔离网络、进程、文件系统等。它们相互独立但共享同一物理硬件资源（如 CPU、内存、磁盘）。

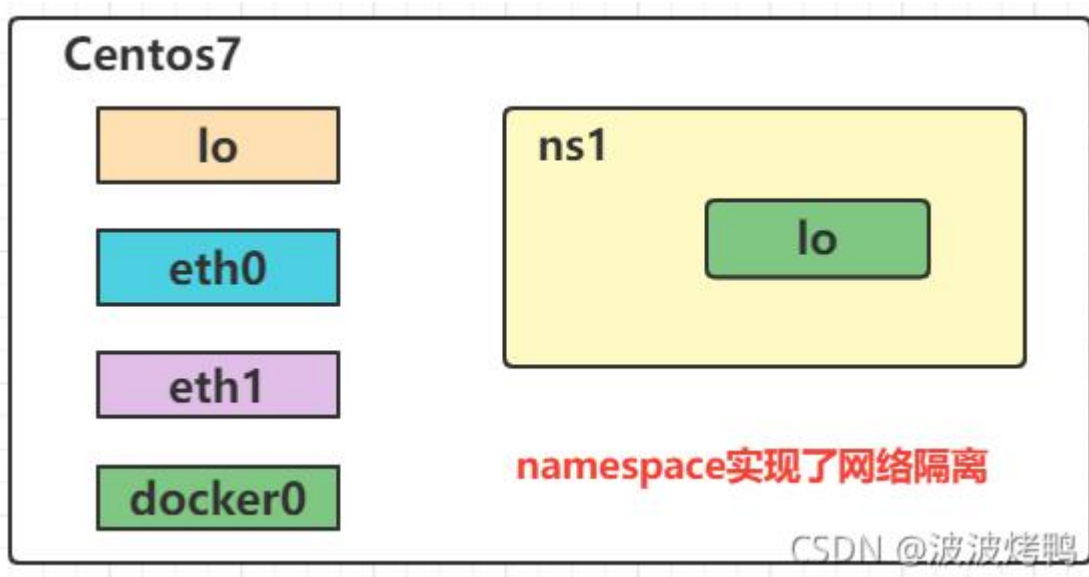


2. 网络隔离 (Network NameSpace)

(1) 概念

Network NameSpace (网络隔离技术) 是 Linux 的 NameSpace 技术 (资源隔离技术) 之一。可以创建独立的网络空间，内置多张网卡，以文件的形式映射在磁盘；

NameSpace 之间是相互隔离的，它们的网卡是无法通信的，相当于两个独立的局域网；



(2) 默认情况

在 Linux 中，网卡运行在默认网络命名空间(Root Network Namespace)中，以文件的形式映射在默认文件夹

```
[root@localhost ~]# ls /sys/class/net
docker0  ens33  lo  virbr0  virbr0-nic
```

网卡

网卡状态有三个：UNKNOWN、UP (开启)、DOWN (关闭)，默认情况下是 DOWN(关闭)。

```
[root@localhost ~]# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:0c:29:29:b2:d9 brd ff:ff:ff:ff:ff:ff
    inet 192.168.124.128/24 brd 192.168.124.255 scope global noprefixroute dynamic ens33
        valid_lft 1200sec preferred_lft 1200sec
    inet6 fe80::4cc8:8d14:2759:71ae/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: virbr0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default qlen 1000
    link/ether 52:54:00:11:82:52 brd ff:ff:ff:ff:ff:ff
```

(3) 自定义网络空间

可以通过命令创建自定义网络空间，添加网卡等设备，并为每一个网卡设置 IP 等信息；

```
[root@localhost ens33]# ip netns list
[root@localhost ens33]# ip netns add ns1
[root@localhost ens33]# ip netns exec ns1 ip a
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
[root@localhost ens33]# ip netns exec ns1 ip link set lo up
[root@localhost ens33]# ip netns add ns2
[root@localhost ens33]#
```

(4) veth pair(Virtual Ethernet Pair)

veth pair 相当于一根虚拟网线，将两个 Network NameSpace 进行连接，并会分别在两端的网络空间创建一张虚拟网卡；

我们需要给这两张虚拟网卡设置同网段且不冲突的 IP，两个网络空间才能访问；

需要注意的是：Network NameSpace 通过 veth pair 连接，只能是点对点，两两互相，从一个网络空间到另一个；

